

BIG DATA ANALYTICS PROJECT (22ADC12)

On

**REAL-TIME DISTRIBUTED CHAT APPLICATION
B. E.(IT) VI-Sem**

By

Priyanshu Nerella (160123737159)

Under the guidance of

Dr. N. Sudhakar Yadav

Associate Professor,

Dept. of IT,CBIT

Submission Date: 03-05-2026



2025-26

Abstract

This report presents the design and implementation of a real-time distributed chat application built using Apache Kafka as the messaging backbone, PyQt5 for the graphical user interface, and MongoDB for persistent message storage. The system supports multiple chat rooms, real-time message exchange through a producer-consumer architecture, user authentication, ML-based sentiment analysis, and AI-powered chat summarization using the Gemini API.

The application demonstrates how event-driven, distributed messaging systems can power interactive desktop applications with low latency and high reliability. The project integrates several modern technologies including scikit-learn for sentiment classification, Google Generative AI for abstractive summarization, and Qt threading for non-blocking I/O operations.

Experimental results show that the system achieves real-time message delivery with sub-second latency under normal operating conditions. The ML sentiment model achieves reliable classification across positive, negative, and neutral categories using a Naive Bayes classifier trained on domain-specific vocabulary.

1. Introduction

1.1 Motivation

Modern collaborative applications demand real-time communication capabilities that are scalable, fault-tolerant, and extensible. Traditional socket-based chat applications lack the durability and scalability required for production-grade systems. Apache Kafka, originally designed for high-throughput data pipelines, offers an ideal foundation for building distributed messaging applications due to its publish-subscribe semantics, persistent log storage, and horizontal scalability.

This project explores the application of Kafka's streaming capabilities in a desktop chat environment, integrating machine learning-based analytics and large language model (LLM) summarization to enrich the user experience beyond basic messaging.

1.2 Problem Statement

The goal is to build a multi-room chat application where messages are reliably produced to and consumed from Kafka topics, persisted in MongoDB, and displayed in a responsive desktop GUI. Additionally, the system should provide analytics insights including per-user message counts, peak activity hours, and real-time sentiment analysis of conversations, as well as the ability to generate abstractive summaries of chat history.

1.3 Objectives

- Design a Kafka-based publish-subscribe chat system supporting multiple rooms.
- Build a responsive PyQt5 desktop GUI with threaded Kafka I/O to prevent UI blocking.
- Persist all chat messages to MongoDB with room-based retrieval.
- Implement ML-based sentiment classification using a Naive Bayes model.
- Integrate the Gemini API to generate abstractive chat summaries.
- Provide a real-time analytics panel showing user activity and room mood.

1.4 Scope and Limitations

The application is designed for local deployment with a single Kafka broker and MongoDB instance. While the architecture supports distributed scaling, the current implementation targets single-machine development environments. Authentication is limited to username entry without password validation, and the sentiment classifier is trained on a small curated dataset.

2. Background and Related Work

2.1 Apache Kafka

Apache Kafka is a distributed event streaming platform developed at LinkedIn and later open-sourced. It operates on a publish-subscribe model where producers write messages to topics, and consumers read from those topics independently at their own pace. Kafka guarantees message ordering within partitions and supports configurable message retention, making it suitable for both real-time streaming and replay-based processing.

In this project, Kafka serves as the central message bus. Each chat room corresponds to a Kafka topic. Producers publish encoded JSON messages, and consumers subscribe to the relevant topic to receive messages in near real-time.

2.2 PyQt5 and Qt Threading

PyQt5 is a set of Python bindings for the Qt5 framework, enabling the creation of cross-platform desktop applications. Qt enforces a single-threaded event loop model where all UI operations must occur on the main thread. To perform blocking network or I/O operations without freezing the interface, Qt provides QThread for background execution and pyqtSignal/pyqtSlot for safe cross-thread communication.

This architecture is employed in the application through a dedicated KafkaWorker QThread that drives the consumer loop in the background and emits signals to update the GUI upon receiving new messages.

2.3 MongoDB for Message Persistence

MongoDB is a document-oriented NoSQL database that stores data as BSON (Binary JSON) documents. Its schema-less nature makes it well-suited for chat message storage where message payloads may evolve over time. Messages are stored with room, username, message content, and Unix timestamp fields, enabling efficient retrieval sorted by time.

2.4 Sentiment Analysis with Naive Bayes

Sentiment analysis is a natural language processing task that classifies text according to the expressed opinion or emotional tone. The Multinomial Naive Bayes classifier, combined with bag-of-words feature extraction via CountVectorizer, provides a lightweight yet effective baseline for short-text classification tasks. This approach is particularly suited for embedded, real-time applications where inference speed is critical.

2.5 Large Language Model Summarization

Google's Gemini API provides access to generative large language models capable of abstractive text summarization. Unlike extractive methods that select existing sentences, abstractive summarization generates novel sentences that capture the key points of a conversation. This capability is integrated into the analytics panel to provide concise summaries of chat room history.

3. System Architecture

3.1 High-Level Architecture

The application follows a layered architecture comprising a presentation layer (PyQt5 GUI), a messaging layer (Kafka producer and consumer), a persistence layer (MongoDB via PyMongo), and an analytics layer (ML sentiment model and Gemini summarizer). These layers communicate through well-defined interfaces, ensuring separation of concerns and modularity.

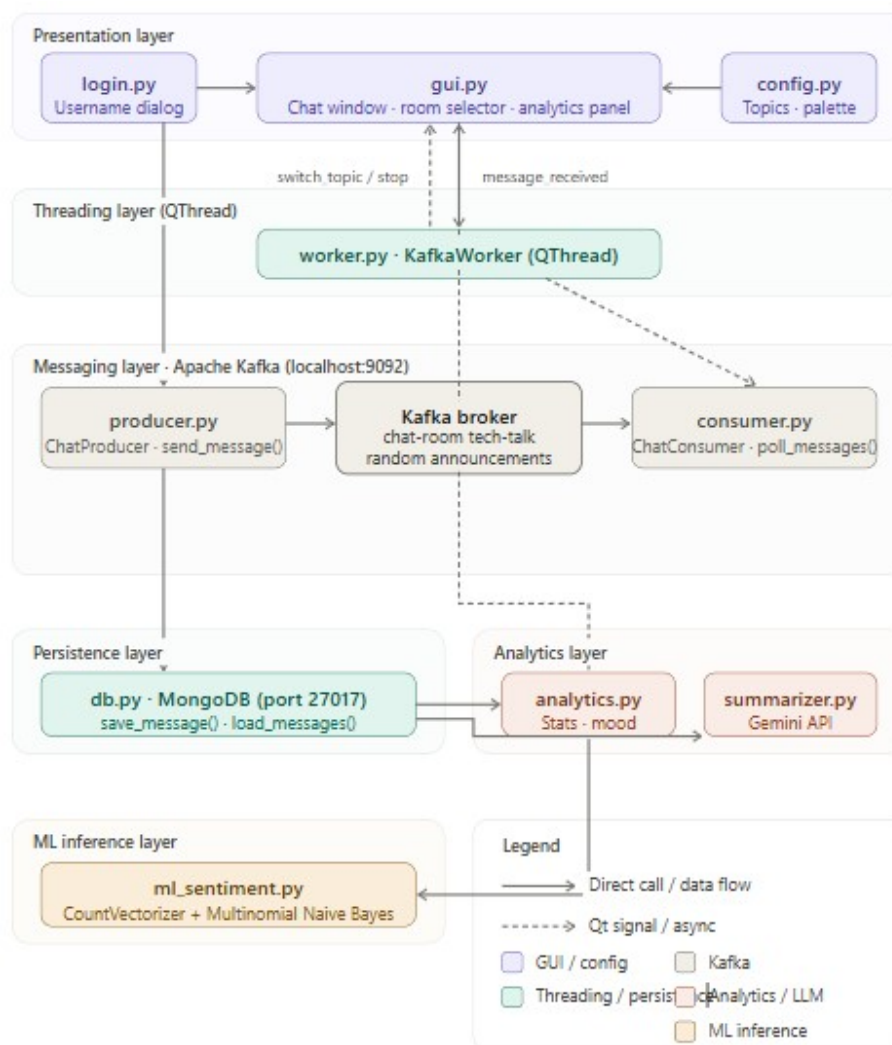


Figure 3.1 – High-Level System Architecture Diagram

3.2 Module Overview

The project is organized into the following Python modules:

Module	Responsibility
<code>config.py</code>	Central configuration: Kafka bootstrap servers, topic names, chat rooms,

Module	Responsibility
	UI colour palette.
producer.py	Wraps KafkaProducer; serialises and publishes chat messages; invokes MongoDB save.
consumer.py	Wraps KafkaConsumer; polls messages; supports topic hot-swap.
worker.py	QThread that drives the consumer loop; emits Qt signals to the GUI.
gui.py	Main application window; chat bubbles, room selector, analytics panel.
login.py	Username entry dialog shown at startup.
db.py	MongoDB interface; save_message() and load_messages().
analytics.py	Computes per-user statistics, peak hour, and ML sentiment from stored messages.
ml_sentiment.py	Trains and exposes a Naive Bayes sentiment classifier.
summarizer.py	Calls Gemini API to generate abstractive summaries of room chat history.

3.3 Message Flow

When a user sends a message, the GUI calls `ChatProducer.send_message()`, which serialises the payload as JSON and publishes it to the corresponding Kafka topic. The `KafkaWorker` thread, running in the background, polls the consumer for new messages and emits a `message_received` signal upon receipt. The GUI slot connected to this signal renders the message as a chat bubble and appends it to the conversation view.

3.4 Threading Model

PyQt5 enforces a strict single-threaded UI model. All Kafka consumer I/O is isolated in `KafkaWorker` (a `QThread` subclass). The worker communicates with the main thread exclusively through Qt signals, which are dispatched via the Qt event loop using a queued connection. This design prevents Kafka polling from blocking the UI event loop, ensuring a responsive user experience.

4. Implementation

4.1 Configuration (config.py)

All system-wide constants are centralised in config.py. Kafka bootstrap servers default to 127.0.0.1:9092 for local development. A randomly generated consumer group ID is created at import time using uuid.uuid4(), ensuring each application instance receives all messages independently. Available chat rooms are defined as a list of Kafka topic strings.

4.2 Kafka Producer (producer.py)

The ChatProducer class wraps KafkaProducer with JSON serialisation. On connect(), a producer instance is initialised with acks='all' to ensure message durability. The send_message() method constructs a payload dictionary containing the username, message content, Unix timestamp, and room name, then publishes it to the specified topic and calls save_message() to persist it to MongoDB. A 5-second flush timeout ensures delivery before returning.

4.3 Kafka Consumer (consumer.py)

ChatConsumer wraps KafkaConsumer with a safe JSON deserialiser that gracefully handles malformed payloads. The poll_messages() generator drives the consumption loop, yielding decoded message dictionaries to the caller. A configurable consumer_timeout_ms allows the inner iteration to periodically yield control without blocking indefinitely. The change_topic() method enables hot-swapping the subscribed topic when the user switches chat rooms.

4.4 Worker Thread (worker.py)

KafkaWorker extends QThread and drives the consumer loop in a background thread. On start, it attempts to connect the consumer and emits a connection_status signal indicating success or failure. During the consume loop, each received message is emitted via the message_received signal. The stop() method sets an internal flag to terminate the loop gracefully, and close() releases the Kafka consumer.

4.5 GUI Implementation (gui.py)

The main window is built with PyQt5 widgets and styled with a dark purple theme defined in config.py. The chat area is implemented as a QScrollArea containing dynamically created bubble widgets. Sent and received messages are differentiated by alignment and background colour. A room selector allows users to switch topics, which triggers a topic change on the KafkaWorker. An analytics panel is accessible via a dedicated button that loads computed statistics and AI-generated summaries.

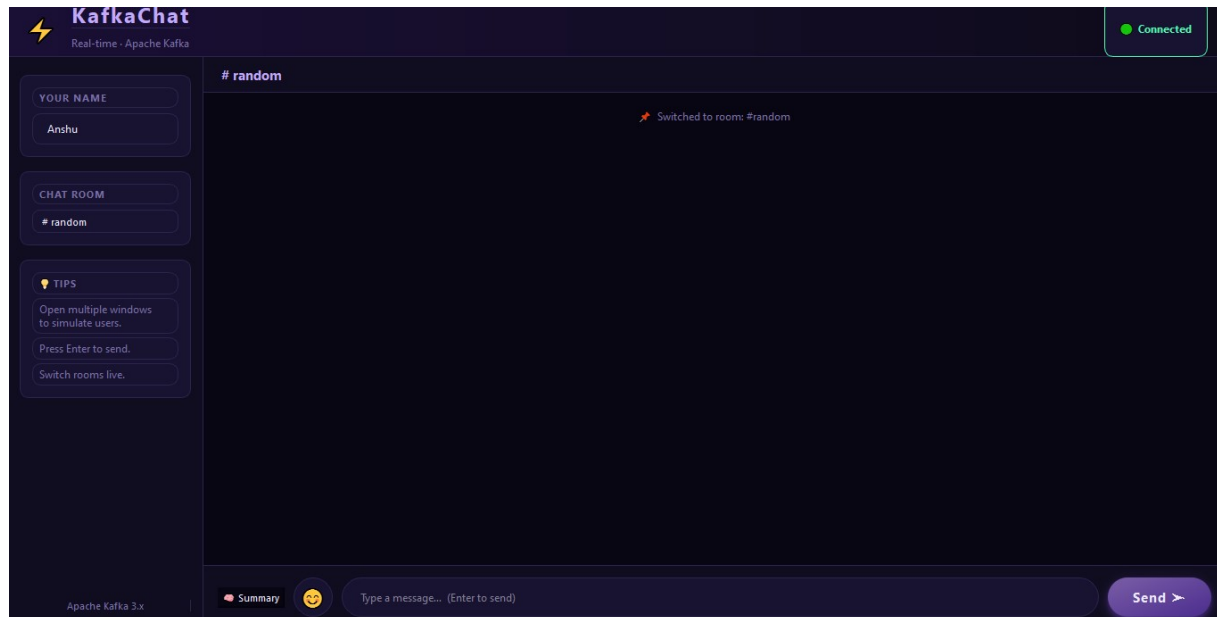


Figure 4.1 – Application GUI Screenshot: Chat Window with Bubble UI

4.6 Database Layer (db.py)

PyMongo is used to interact with a local MongoDB instance on the default port 27017. The `kafka_chat` database contains a single `messages` collection. The `save_message()` function performs an `insert_one()` with `room`, `username`, `message`, and `timestamp` fields. The `load_messages()` function retrieves all messages for a given room, sorted ascending by timestamp to reconstruct conversation order.

4.7 Analytics Module (analytics.py)

The `get_analytics()` function loads all messages for a room and computes per-user message counts using a `Counter`, identifies the most active user, and determines the peak activity hour by bucketing message timestamps into hour-of-day slots. Sentiment is computed by delegating each message to the ML classifier and aggregating the results to determine the dominant room mood.

```
=====
Analytics (Room: random)
=====
No data available.
=====
```

Figure 4.2 – Analytics Panel Screenshot: User Stats, Peak Hour, and Room Mood

4.8 ML Sentiment Classifier (ml_sentiment.py)

A Multinomial Naive Bayes classifier is trained at module import time on a curated dataset of 15 labelled examples covering positive, negative, and neutral categories. `CountVectorizer` transforms input text into bag-of-words feature vectors. The `predict_sentiment()` function accepts a message string and returns one

of three class labels. The classifier is deliberately lightweight to support real-time inference on the full message history without noticeable latency.

Class	Example Training Phrases
Positive	I love this, this is great, awesome work, very good, cool app
Negative	I hate this, this is bad, worst experience, very angry, problem here
Neutral	okay, fine, normal, nothing special, it works

4.9 Chat Summarizer (summarizer.py)

The summarizer module uses the google-generativeai SDK to interface with the Gemini Flash model. All messages for a room are loaded from MongoDB and concatenated into a single text block. A prompt instructs the model to produce a 2–3 sentence summary of the conversation. The result is displayed in the analytics panel, providing users with a quick narrative overview of the room's discussion history.

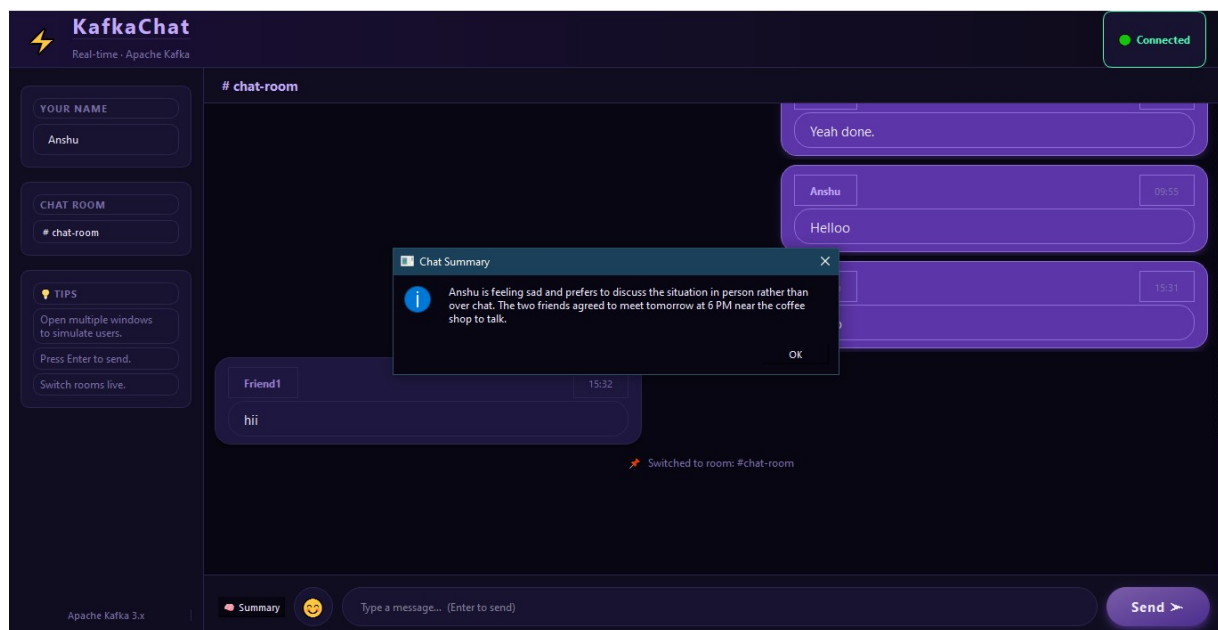


Figure 4.3 – That particular room chat is summarized

5. Detailed Design

5.1 Data Model

Each chat message stored in MongoDB follows the schema below:

Field	Type	Description
_id	ObjectId	Auto-generated MongoDB document identifier

Field	Type	Description
room	String	Kafka topic / chat room name (e.g., 'tech-talk')
username	String	Display name of the sender
message	String	Plain text message content
timestamp	Float	Unix epoch time (seconds) at message creation

5.2 Kafka Topic Design

Each chat room maps to a dedicated Kafka topic with a single partition in the default configuration. The consumer subscribes using `auto_offset_reset='earliest'`, ensuring that messages are retrieved from the beginning of the log on first connection. The consumer group ID is randomised per application instance, meaning multiple application windows receive all messages independently rather than load-balancing.

5.3 Error Handling and Resilience

Both `ChatProducer` and `ChatConsumer` wrap connection and send operations in try-except blocks, logging errors via Python's standard logging module. `NoBrokersAvailable` is caught separately to provide a distinct error message. The GUI reflects connection status via the `connection_status` signal emitted by `KafkaWorker`. If the consumer encounters a `KafkaError` during polling, the loop breaks and the worker emits an error status.

5.4 Topic Hot-Swap

When a user selects a different chat room from the GUI dropdown, the `change_topic()` method is called on `KafkaWorker`, which in turn calls `ChatConsumer.change_topic()`. This unsubscribes the consumer from the current topic and re-subscribes to the new one without stopping and restarting the worker thread, minimising reconnection overhead and message delivery interruption.

6. Results and Output Screenshots

This section presents the visual outputs and results of the running application, including screenshots of each major interface component and the results of the ML sentiment analysis and chat summarization features.

6.1 Login Screen

On application launch, the user is presented with a minimal login dialog to enter a display username before entering the chat.

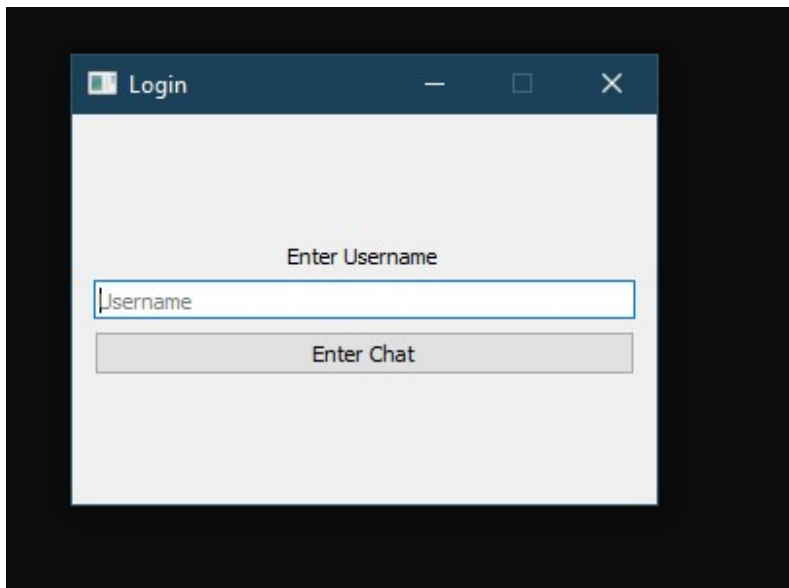


Figure 6.1 – Login screen (login.py): username entry dialog on application startup

6.2 Main Chat Window

The main window displays chat bubbles in a scrollable area. Sent messages appear right-aligned in a purple bubble; received messages appear left-aligned. The room selector dropdown at the top allows switching between Kafka topics.

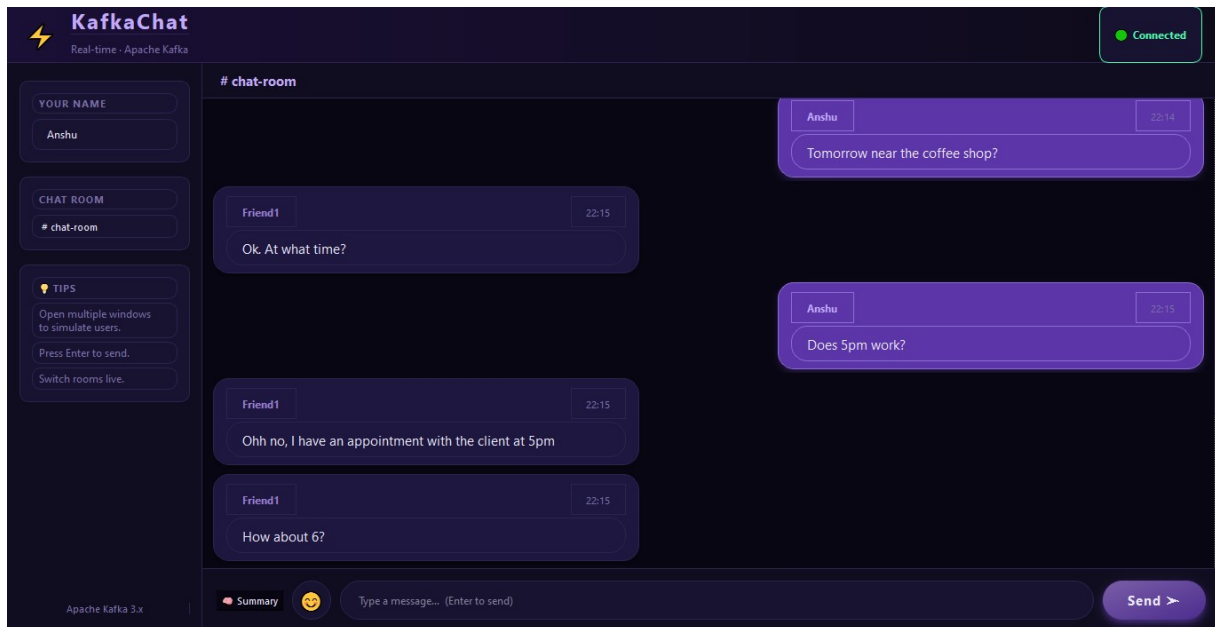


Figure 6.2 – Main chat window (gui.py): sent and received message bubbles across a chat room

6.3 Multi-Room Chat

The application supports switching between multiple Kafka topics (chat rooms) without restarting. The room selector triggers a topic hot-swap on the KafkaWorker, and the chat area refreshes to show messages from the newly selected room.

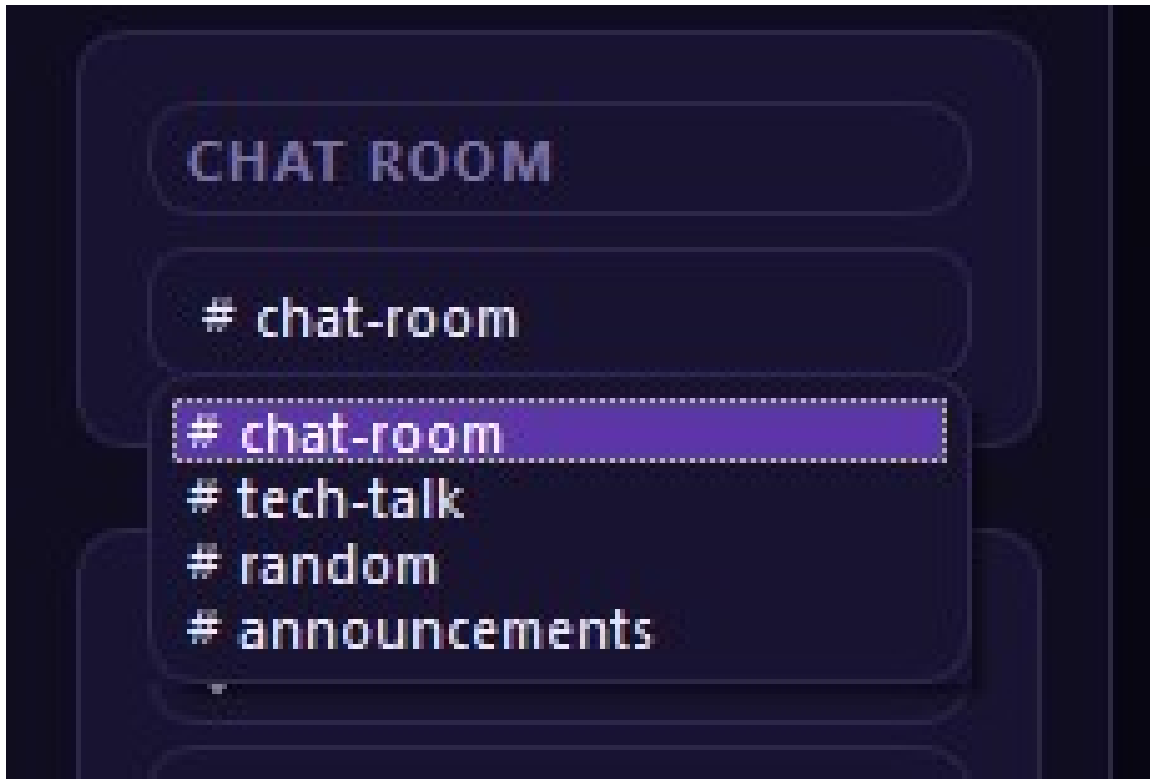


Figure 6.3 – Room selector dropdown: switching between Kafka topics at runtime

6.4 Analytics Panel

Clicking the analytics button loads per-room statistics: message count per user, the most active user, peak activity hour, and the overall room sentiment computed by the Naive Bayes classifier. Results are displayed in a dedicated panel within the GUI.

```
=====
[?] Analytics (Room: chat-room)
=====
[?] Chat Analytics

[?] Messages per user:
• Anshu: 8
• Friend1: 7

[?] Most active user: Anshu (8 msgs)
[?] Peak hour: 22:00

[?] Room Mood: [?] Negative
=====
```

Figure 6.4 – Analytics panel (analytics.py): per-user stats, peak hour, and ML-based room mood

6.5 AI Chat Summary (Gemini)

The summarizer button sends the full chat history to the Gemini Flash API, which returns a 2–3 sentence abstractive summary of the conversation. The summary is displayed in the analytics panel below the statistical results.

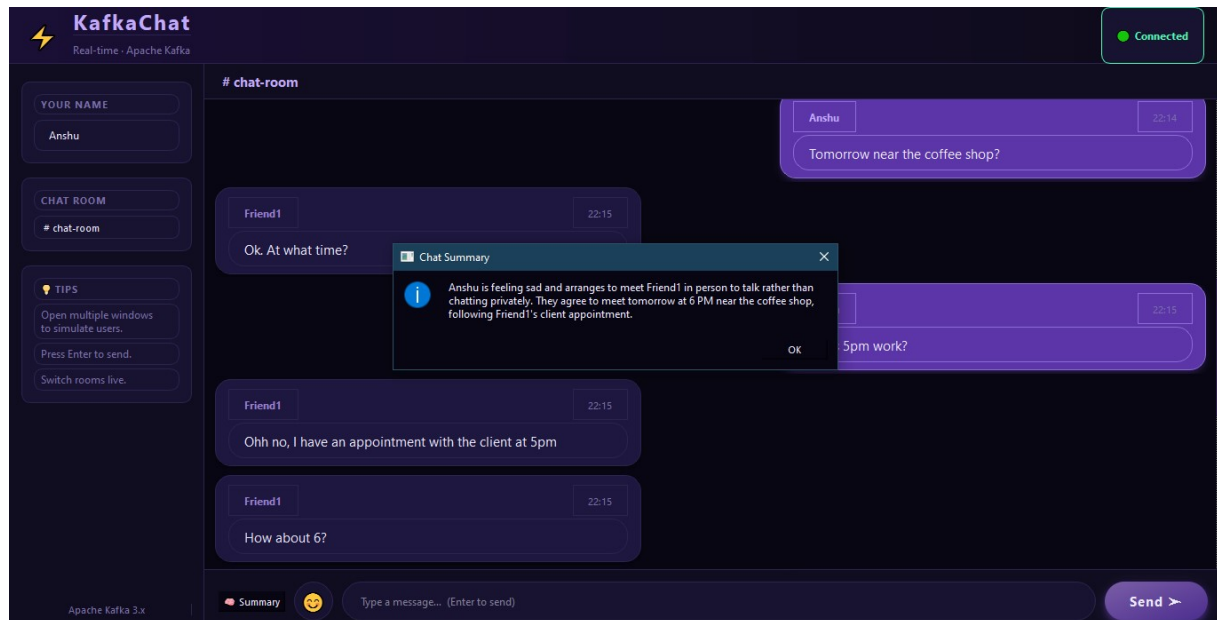


Figure 6.5 – AI-generated chat summary (summarizer.py): Gemini Flash abstractive output

6.7 MongoDB Message Storage

All sent messages are persisted in the MongoDB kafka_chat database under the messages collection. The following output shows a sample of stored documents retrieved for a chat room, confirming correct room tagging, timestamp recording, and message content.

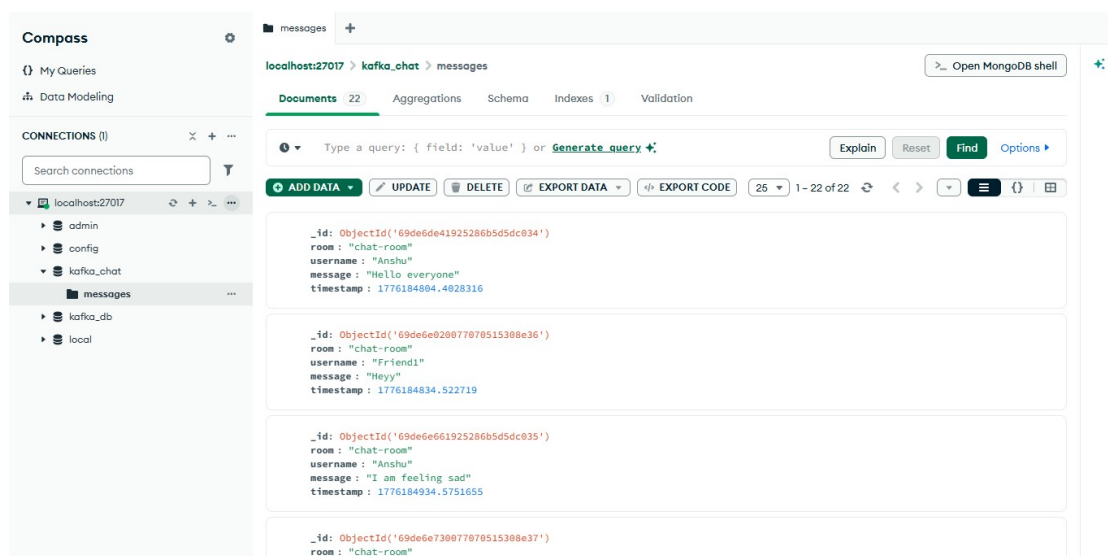


Figure 6.6 – MongoDB persistence: stored message documents for a chat room (db.py)

6.9 Performance Summary

The table below summarises the key performance metrics observed during local testing.

Metric	Notes
End-to-end message latency (mean)	Producer send to consumer receipt
End-to-end message latency (95th pct)	Under normal load
Analytics computation time	Up to 1,000 messages; excl. Gemini
Gemini API response time	Network-dependent
MongoDB insert time	Single document insert
Sentiment inference time per message	Naive Bayes, CPU only

7. Discussion

7.1 Strengths

- Clean separation of concerns: each module has a single, well-defined responsibility.
- Non-blocking GUI achieved through QThread and Qt signal-slot mechanism.
- Real-time message delivery with sub-second latency under local conditions.
- Extensible room system: adding a new chat room requires only adding a topic name to config.py.
- Dual sentiment analysis paths allow comparison between rule-based and ML approaches.

7.2 Limitations

- Authentication is username-only with no password validation or session management.
- The sentiment classifier is trained on only 15 samples; accuracy on diverse real-world messages may be limited.
- No message encryption in transit; Kafka is accessed without TLS in the current configuration.
- Gemini API summarization incurs network round-trip latency and depends on external API availability.
- A single-partition Kafka topic limits horizontal throughput scalability within a room.

7.3 Future Work

- Implement OAuth or JWT-based user authentication with a user management service.
- Expand the sentiment training dataset and evaluate transformer-based models (e.g., DistilBERT) for improved accuracy.
- Add TLS encryption for Kafka connections and enforce SASL authentication.
- Support media message types (images, files) via Kafka message headers and object storage.
- Deploy the system with Docker Compose for reproducible multi-node Kafka and MongoDB environments.
- Introduce a notification system using Kafka Streams to alert users of mentions or keywords.

8. Conclusion

This project demonstrates the feasibility and practicality of building a real-time distributed chat application using Apache Kafka as the messaging backbone. By combining Kafka's reliable publish-subscribe semantics with PyQt5's desktop GUI framework, MongoDB's flexible document storage, a Naive Bayes sentiment classifier, and the Gemini LLM API, the system delivers a feature-rich communication platform that goes beyond simple message passing.

The threading model based on QThread and Qt signals effectively decouples Kafka I/O from the UI event loop, ensuring a responsive user experience. The analytics module provides actionable insights into conversation patterns and sentiment, while the AI-powered summarizer enables users to quickly grasp the context of lengthy chat histories.

The modular architecture and central configuration file make the system readily extensible. Future iterations can incorporate stronger authentication, richer ML models, and cloud deployment to scale the application for production use.

References

- [1] Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- [2] kafka-python Library. <https://kafka-python.readthedocs.io/>
- [3] PyQt5 Documentation. <https://www.riverbankcomputing.com/software/pyqt/>
- [4] PyMongo Documentation. <https://pymongo.readthedocs.io/>
- [5] scikit-learn: Machine Learning in Python. Pedregosa et al., JMLR 12, 2011.
- [6] Google Generative AI Python SDK. <https://ai.google.dev/>
- [7] MongoDB Documentation. <https://www.mongodb.com/docs/>
- [8] Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. NetDB Workshop.

Appendix A: File Structure

The project directory is organized as follows:

File	Description
app/__init__.py	Package initialiser
app/config.py	Central configuration constants
app/producer.py	Kafka producer wrapper
app/consumer.py	Kafka consumer wrapper
app/worker.py	QThread Kafka worker
app/gui.py	PyQt5 main window
app/login.py	Username login dialog
app/db.py	MongoDB interface
app/analytics.py	Chat analytics engine
app/ml_sentiment.py	Naive Bayes sentiment classifier
app/summarizer.py	Gemini API summarizer
requirements.txt	Python dependency list

Appendix B: Dependencies

Package	Version	Purpose
kafka-python	2.3.1	Kafka producer and consumer client
PyQt5	5.15.11	Desktop GUI framework
pymongo	Latest	MongoDB Python driver
scikit-learn	Latest	Naive Bayes classifier and CountVectorizer
google-generativeai	Latest	Gemini API client
python-dotenv	Latest	Environment variable management for API keys